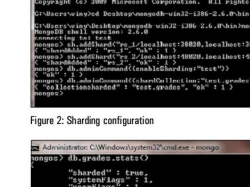


Figure 2: Sharding configuration


Figure 3: Sharding status

```
sh.addShard("rs_1/localhost:38020,localhost:38021,localhost:38022")
sh.addShard("rs_2/localhost:48020,localhost:48021,localhost:48022")
```

These two commands will add the two shards that were configured earlier. Enable sharding for the test database using the following command:

```
db.adminCommand({enableSharding:"test"})
```

Finally, enable sharding on the grades collection under the test database by using the code shown below:

```
db.adminCommand({shardCollection:"test.grades",key:{student_id:1}})
```

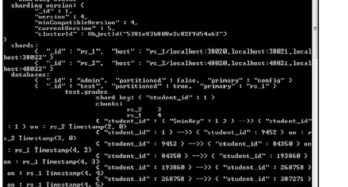


Figure 4: Data distribution between shards

In this instance, *student_id* is our shard key, which is used to distribute data among shards. Do note that this collection does not exist right now. To use sharding, we need an index on the shard key. Here, the collection does not exist and the index will be created automatically. But if you have data in your collection, then you'll have to create an index on your shard key. You can verify your sharding status by using the *sh.status()* command.

We now have our sharded environment up and running, and it's time to see how it works by inserting data in the grades collection. So type the following simple javascript code snippet to insert data:

```
for (var I = 1; I <= 500000; i++) db.grades.insert( {
student_id : I,Name:"Student"+I } )
```

Now if you type *db.grades.stats()*, you'll find that some of these records are stored in the *rs_1* shard and others are stored in the *rs_2* shard.

You can see that out of 500,000 records inserted, 377,819 are stored in shard *rs_1* while the remaining 122181 go to shard *rs_2*. If you fire the *sh.status()* command, you can figure out how data is distributed between these two shards.

In Figure 4, you can see that student IDs ranging from 1 - 9452 and 387,271 - 500,000 (represented by *$maxKey*) are stored on *rs_2* while the remaining data is on *rs_1*.This data distribution is transparent to the end user; data storage and retrieval is handled by the *mongos* router.

Try to configure and explore the overall process a few more times so that you can feel more confident in implementing sharding in MongoDB. END

**By: Vinayak Pandey**

The author is an experienced database developer, with exposure to various database and data warehousing tools and techniques, including Oracle, Teradata, Informatica PowerCenter and MongoDB.